

FEDGKD: Toward Heterogeneous Federated Learning via Global Knowledge Distillation

Dezhong Yao , Member, IEEE, Wanning Pan , Yutong Dai , Yao Wan , Xiaofeng Ding , Chen Yu ,
Hai Jin , Fellow, IEEE, Zheng Xu , and Lichao Sun 

Abstract—Federated learning, as one enabling technology of edge intelligence, has gained substantial attention due to its efficacy in training deep learning models without data privacy and network bandwidth concerns. However, due to the heterogeneity of the edge computing system and data, many methods suffer from the “client-drift” issue that could considerably impede the convergence of global model training: local models on clients can drift apart, and the aggregated model can be different from the global optimum. To tackle this issue, one intuitive idea is to guide the local model training by global teachers, *i.e.*, past global models, where each client learns the global knowledge from past global models via adaptive knowledge distillation techniques. Inspired by these insights, we propose a novel approach for heterogeneous federated learning, FEDGKD, which fuses the knowledge from historical global models and guides local training to alleviate the “client-drift” issue. In this paper, we evaluate FEDGKD through extensive experiments across various CV and NLP datasets (*i.e.*, CIFAR-10/100, Tiny-ImageNet, AG News, SST5) under different heterogeneous settings. The proposed method is guaranteed to converge under common assumptions and outperforms the state-of-the-art baselines in the non-IID federated setting.

Index Terms—Heterogeneous federated learning, non-IID, knowledge distillation, edge intelligence.

I. INTRODUCTION

EDGE intelligence [1], [2] has recently emerged as a promising solution that brings deep learning models closer to data sources to significantly reduce response time.

Manuscript received 28 June 2022; revised 1 March 2023; accepted 27 July 2023. Date of publication 15 September 2023; date of current version 22 December 2023. This work was supported in part by the National Key Research and Development Program of China under Grant 2021YFB1714600 and in part by the National Natural Science Foundation of China under Grants 62072204, 62032008, and 62102157. The work of Lichao Sun and Yutong Dai was supported by the National Science Foundation under Grant CRII-2246067. The computation was completed in the HPC Platform of Huazhong University of Science and Technology and was supported by the National Supercomputing Center in Zhengzhou. Recommended for acceptance bybreak M. Annavaram. (Corresponding authors: Dezhong Yao; Lichao Sun.)

Dezhong Yao, Wanning Pan, Yao Wan, Xiaofeng Ding, Chen Yu, and Hai Jin are with the National Engineering Research Center for Big Data Technology and System, Service Computing Technology and System Lab, Cluster and Grid Computing Lab, School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan 430074, China (e-mail: dyao@hust.edu.cn; pwn@hust.edu.cn; wanyao@hust.edu.cn; xfding@hust.edu.cn; yuchen@hust.edu.cn; hjin@hust.edu.cn).

Yutong Dai and Lichao Sun are with Lehigh University, Bethlehem, PA 18015 USA (e-mail: yud319@lehigh.edu; lis221@lehigh.edu).

Zheng Xu is with Google Research, San Francisco, San Francisco, CA 94043 USA (e-mail: xuzheng@google.com).

Digital Object Identifier 10.1109/TC.2023.3315066

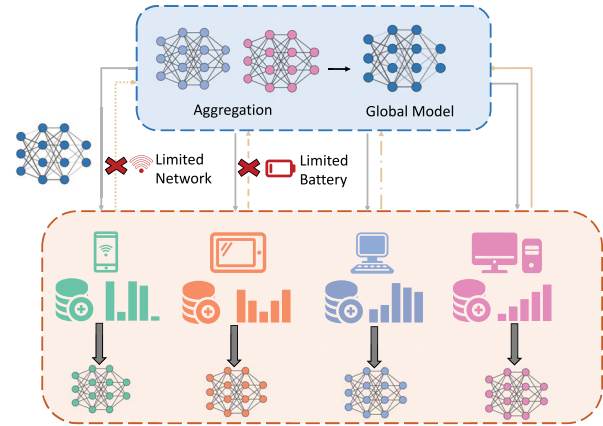


Fig. 1. Two main challenges in federated learning. *Data heterogeneity*: a large difference among the data distribution of clients, and *client unreliability*: some of the clients are unable to upload their local models due to limited network and battery resources during the training.

Benefiting from the fast response and bandwidth relief, edge intelligence has become a popular paradigm to support time- and quality-sensitive applications, such as autonomous vehicles [1], unmanned aerial vehicle sensing [3], anomaly detection [4], and healthcare [2]. One of the enabling techniques of edge intelligence is the privacy-preserving learning method known as *Federated Learning* (FL), which is an effective distributed machine learning framework.

Federated learning (FL) [5], [6] is an effective distributed machine learning framework, as shown in Fig. 1, where local models perform local training on participating edge devices (*clients*) and upload the update (gradients/parameters of models) to the central coordinator (*server*). Different from traditional distributed machine learning systems [7], [8], [9], FL aims to enable multiple clients to collaboratively learn a global model without sharing their local data. In the classic federated learning algorithm FedAvg [10], the clients perform multiple epochs of *Stochastic Gradient Descent* (SGD) on their own datasets, and then the updated models are submitted to the server for aggregation via model averaging. This method greatly reduces communication costs in environments where clients’ data are *Independent and Identically Distributed* (IID).

Due to the complexity of the edge environment, two critical challenges have attracted growing attention in federated learning [5], [11], [12]: (1) *data heterogeneity*, and (2) *client unreliability*. The challenges are shown in Fig. 1. More specifically,

TABLE I
COMPARISON OF FEDERATED LEARNING APPROACHES

Method	Local Information	Proxy Data	Model Modification
FedAvg [5]	✗	✗	✗
FedProx [15]	✗	✗	✗
FedDistill [17]	✓	✗	✗
FedGen [18]	✓	✗	✗
FedDF [13]	✗	✓	✗
MOON [19]	✗	✗	✓
FEDGKD (Ours)	✗	✗	✗

data heterogeneity refers to the issue that data distributions on clients are often non-independent and identically distributed (non-IID). Since the local data on each client is not sampled from the global joint distribution of all clients [5], the local objectives are different and may not share common minimizers. Even if every communication round starts from the same global model, clients' local models will *drift* towards the minima of their local objectives, thus the aggregated global model may not be the optimum of the global objective. Such “*client-drift*” phenomenon not only degrades the performance but also increases the number of communication rounds. Therefore, it is desirable to explicitly handle the non-IID issue. Client unreliability means that there is only a small portion of clients participating in each communication round because the hardware resources of participating devices are limited [1]. The low participation rate exacerbates the impact of the non-IID issue and results in poor performance of the global model in FL.

Currently, many efforts have been committed to addressing the non-IID issue and can be classified into two categories. (1) The first line of work tackles this issue by improving the model aggregation scheme. Specifically, with the help of unlabeled proxy datasets, FedDF, etc. [13], [14] employ the ensemble knowledge distillation to enrich the global model. However, the requirement of proxy data limits its deployment in real FL scenarios. Besides, these approaches do not fully address the client-drift problem, since they only refine the global model. (2) The second line of work approaches this issue by regularizing local training. Specifically, SCAFFOLD [11], FedProx [15], and FedDyn [16] require the local clients' models cannot be too far away from the global model in the *parameter space*, which may fail to work effectively in some FL scenarios [5]. FedDistill [17], FedGen [18], and MOON [19] regularizes training in the *feature space* by requiring the extracted features (logits) between the local model and the global model are similar. However, FedDistill needs to share the clients' logits to the server, and FedGen needs to share the clients' label counts, which leads to the risk of privacy leakage. MOON proposes to use contrastive loss for regularization, but it requires modifying the structure of models. We summarize the limitations of the aforementioned methods in Table I.

We aim to address the above challenges without introducing any privacy risks or model structure modification requirements. In this paper, we propose a Federated Learning framework via Global Knowledge Distillation, named FEDGKD¹, which

¹The source code is available at: <https://github.com/CGCL-codes/FedGKD>

utilizes knowledge distillation and self-ensemble [20], [21] to tackle the client-drift issue in feature space. At each communication round, FEDGKD ensembles the historical global models in the server buffer by averaging the model parameters. Then the ensemble global model is broadcast to the clients and serves as a guide (teacher) for local training on knowledge distillation. We also develop FEDGKD-VOTE, by broadcasting the historical global models in the server buffer to the clients, to ensemble their predictions on the client side. This self-distillation method has shown that it can be used for label smoothing and improving the generalization performance of model [22]. Additionally, knowledge distillation ensembles the local models and global models and distills the knowledge to local models, so that local models can better capture different features of data. Furthermore, clients often vary in the computation capabilities in practice [23], and recent work proposes to apply a heterogeneous model in federated learning [13], [24], [25]. The aforementioned methods based on parameter space optimization [11], [16] fail to work in this scenario due to the assumption that model parameters remain unchanged, while FEDGKD is compatible with the existing heterogeneous model framework.

The primary contributions of the paper are as follows.

- We introduce an ensemble-based knowledge distillation technique to transfer the information of historical global models to local model training, which regularizes local models in feature space without model modifications.
- We provide a general and simple method FEDGKD for federated knowledge distillation, which does not require additional information sharing or proxy data. FEDGKD is compatible with many privacy protection methods and orthogonal to different aggregation schemes. Furthermore, the training overhead of FEDGKD is at the same complexity level as FedAvg, and FEDGKD reduces the communication cost to reach target accuracy.
- We conduct extensive experiments and analysis on various datasets (i.e. CIFAR-10/100, Tiny-ImageNet, Ag News, and SST-5) and settings (i.e. different levels of non-IID data distribution and participation ratio) to validate the effectiveness of the proposed FEDGKD, with comprehensive comparison to several state-of-the-art methods.

II. BACKGROUND AND MOTIVATION

In this section, we first introduce several basic concepts of federated learning and knowledge distillation. Then we present the motivation for using knowledge distillation and ensemble mechanisms to address the client-drift issue.

A. Federated Learning

Federated Learning (FL) is a communication-efficient distributed learning framework where a subset of clients performs local training before aggregation [5], which can train a machine learning model without sharing clients' private data [26], therefore the data privacy will be preserved. In FL, one critical problem is on handling the unbalanced, non-independent and identically distributed (non-IID) data, which is very common in the real world [27].

Suppose there are N clients $C_1, \dots, C_N$, where client C_k stores a local dataset D_k with a distribution $\mathcal{D}_k$. In general, FL aims to learn a global model weight $\mathbf{w}$ over the dataset $\mathcal{D} = \bigcup \{D_k\}_{k=1}^{n_K}$, where n_K is the number of participants in each training round. The objective of FL is to solve the following optimization problem [5]:

$$\min_{\mathbf{w} \in \mathbb{R}^d} f(\mathbf{w}) := \sum_{k=1}^K p_k F_k(\mathbf{w}), \quad (1)$$

where $p_k = n_k / \sum_{k=1}^K n_k$, $n_k = |D_k|$ and $F_k(\mathbf{w}) = \frac{1}{n_k} \sum_{i=1}^{n_k} \mathcal{L}(h_k(\mathbf{w}; x_{ki}), y_{ki})$ is the local objective function at the k -th client. Here, $h_k: \mathbb{R}^d \times \mathbb{R}^{d'} \rightarrow \mathbb{R}^C$ represents the local model and $\mathcal{L}: \mathbb{R}^C \times \mathbb{R}^C \rightarrow \mathbb{R}_+$ is the loss function. Moreover, (x_{ki}, y_{ki}) corresponds to a sample point from the k -th client for all $i \in [n_k]$ and $k \in [K]$.

This work aims to optimize the convergence speed of the global model in non-IID environments in FL and to improve the accuracy of FL in this scenario. The ultimate goal is to reduce the communication cost of FL and increase the system's utility. To overcome the non-IID issues, many studies have focused on regularization of client updates, Li et al. [15] add a proximal term in the local objective, and Karimireddy et al. [11] control variate to correct the client-drift. Acar et al. [16] propose a dynamic regularization to ensure the local optima is consistent. Li et al. [19] use the model-level contrastive learning method to correct the local training. On the server side, there have been many efforts to address the non-IID problem. Hsu et al. [28] propose to optimize model update using momentum, Reddi et al. [29] propose an adaptive optimizer in the federated learning, and Wang et al. [30] propose optimizing model aggregation through neuron matching and averaging. FEDGKD mainly focuses on client-side optimization and is orthogonal to the aforementioned methods. Yoon et al. [31] share the average of local data among clients for data augmentation while leaking users' data privacy. Zhang et al. [32] improve model aggregation through adversarial training on the server, but this method requires leaking clients' label information to stabilize adversarial training. Another research direction is personalized federated learning, which focuses on improving the model's performance on clients' own datasets in Non-IID scenarios. Li et al. [33] improve the accuracy of personalized models by not aggregating the Batch Normalization (BN) layers, while it cannot be applied to all models, such as the Transformer [34]. Tan et al. [35] study proto aggregation at the embedding layer, and the model's decision layer only participated in local training. Chen et al. [36] study personalized federated learning based on graph structural information. They have a different goal from this paper.

B. Knowledge Distillation

Knowledge Distillation (KD) is referred to as a teacher-student paradigm in that a cumbersome but powerful teacher model transfers its knowledge through distillation to a lightweight student model [37]. KD aims to minimize the discrepancy between the outputs from the teacher model $\mathbf{w}^T$

and the student model $\mathbf{w}^S$ with a dataset D . The discrepancy could be measured by Kullback-Leibler (KL) divergence:

$$\min_{\mathbf{w}^S} \mathbb{E}_{x \sim \mathcal{D}} [h(\mathbf{w}^T; x) || h(\mathbf{w}^S; x)], \quad (2)$$

where h denotes the model function parameterized by $\mathbf{w}$.

KD has shown great potential in FL tasks. First, many studies focus on the data heterogeneity (non-IID) problem. Seo et al. [17] perform KD to refine the local training by sharing local logits. Zhu et al. [18] propose FedGen to train a generator on a server by sharing the local label count and regularizing the local classifier by training on the generated latent features. While the additional information-sharing methods may face the risk of privacy leakage. Lin et al. [13] propose FedDF to use the averaged logits of local models on proxy data for aggregation. However, the performance of the distillation can be affected by the distribution of proxy data. Although the efficacy of model aggregation is improved with the additional proxy dataset, the biased local models still affect the quality of the ensemble. It is worth noting that our distillation method differs from FedGen and FedDF. FedGen applies regularization by matching the local model outputs to their corresponding hard labels obtained from random sampling, while FedDF performs an ensemble of different local models through knowledge distillation. FEDGKD regularizes local models by matching their outputs to the global ensemble model (i.e., self-distillation) on the client side. Second, some work aims to reduce communications costs. Li et al. [38] require to pre-train the model on a public labeled dataset, and the local models learn consensus through averaged logits on the public dataset. He et al. [39] offload the computation of neural architecture to the server by combining knowledge distillation and split learning. Li et al. [40] propose to utilize knowledge transfer on one-shot federated learning. The above studies either need a pre-designed proxy dataset or require clients to send their private information to the server.

C. Motivation

It is commonly known that deep learning models benefit a lot from data with multiple features [41]. Under non-IID scenarios, local models often fail to capture the comprehensive features due to training on skewed data. For example, in an image of a car, we can classify it as a car by its features like wheels, windows, and headlights. Given a model trained on car and airplane images, the features of headlights might be ignored. In Fig. 2, we present the *Class Activation Maps* (CAMs) of different models on a car image, which represent the features that models pay attention to. Fig. 2(f) indicates the global model, and Fig. 2(b)–(e) indicate the local models of *Client 1* to *Client 4*, respectively. As shown in Fig. 2(d), although there are sufficient car images on *Client 3*, the local model still fails to capture the comprehensive features of the car. Besides, we can find that the global model is able to extract better feature representation. Obviously, the global model shows better generalization performance. Thus we consider using knowledge distillation to steer the global model to guide the local model training so that local models can better capture the multi-view of data. We discuss why knowledge distillation enhances the feature

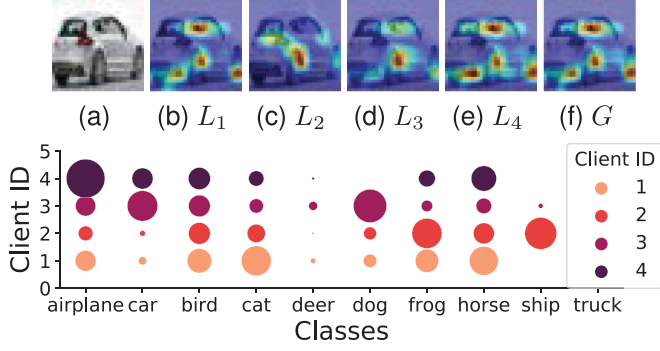


Fig. 2. Top: The class activation maps (CAMs) of global model and local models on a car image. Bottom: Visualization of the samples per class allocated to clients. The size of the dot indicates the number of data.

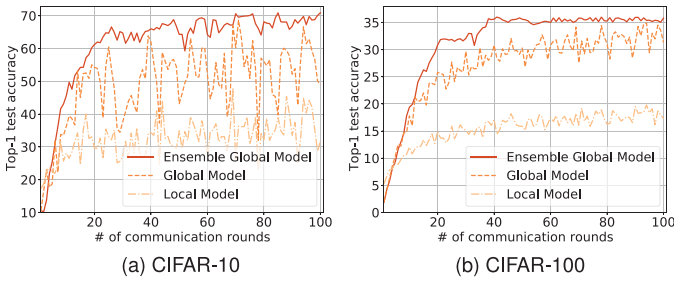


Fig. 3. Performance of ensemble global models, global model of FedAvg, and participated local models under low participant ratio.

extraction capabilities as follows: Supposing $\mathcal{M}_{L_k}, \mathcal{M}_G$ denote the features learned by local model L_k and global model G , where $\mathcal{M}_{L_k}$ come from the local objectives (e.g. cross entropy loss). The local model L_k can learn a larger set of features $\mathcal{M}_{L_k} \cup \mathcal{M}_G$ by matching the output of model G .

However, as shown in Fig. 3, we find that the performance of the global model may fluctuate when there are only a few participants. In this case, the global model is not the best teacher model. We further evaluate the performance of historical global models ensemble and find that it shows better performance than the global model of FedAvg, as shown in Fig. 3. Hence, we regard an ensemble of global models as our guiding teacher model.

III. PROPOSED FEDGKD

We present an overview of the FEDGKD in Fig. 4 and Algorithm 1. Our primary objective is to address the challenge of local model drift by employing global model distillation and assembling the resultant global models to achieve a comprehensive representation.

We implement FEDGKD for the client-server model. At each communication round, the server selects a small portion of clients as participants, and sends the aggregated global knowledge to the selected clients (*Step 1*). In this part, we develop two versions of FEDGKD, and the detailed differences are introduced in the subsequent sections. The clients receive the global knowledge and utilize it to guide their local training (*Step 2*). We address the client-drift issue mainly by regularizing local

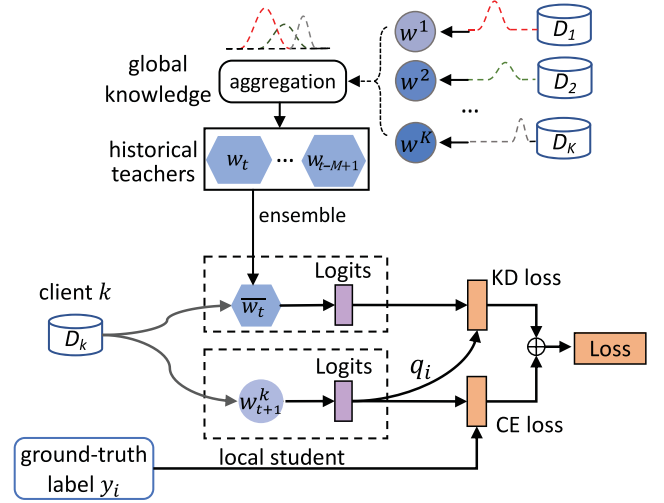


Fig. 4. An overview of FEDGKD: the ensemble model $\bar{w}_t$ is aggregated from historical global models in the server buffer. The ensemble model is then sent to the sampled client, whose global knowledge is distilled to local models for more comprehensive features.

training through a knowledge distillation mechanism. Subsequently, the clients upload their local models, and the server aggregates the local models into a global model and saves the global model to its buffer (*Step 3*). Due to the client unreliability problem, the global model can only achieve good performance on a small portion of dataset (the data held by selected clients). FEDGKD ensembles the historical global models stored in the server buffer (*Step 4*), and treats them as global knowledge for a more comprehensive view of the data. Note that, when we set the buffer size as 1, the global knowledge is equivalent to the global model.

Firstly, we present an initial approach that employs the most up-to-date global model for overseeing local training. Subsequently, we expand upon this approach by forming an ensemble of past global models and leveraging this ensemble for guidance.

A. Global Knowledge Distillation

We modify local training by adding the knowledge distillation loss to transfer global knowledge to clients. Suppose w_t is the parameter of the global model at round t , x is the input of the model, and the dimension of target y is D . Let $h(w, x)$ be the predicted probabilities obtained from the logit vector $z(w, x)$ output by the model w . As suggested by Hinton et al. [42], we utilize the temperature scaling τ to soften the probabilities for better distillation effect:

$$h(w, x) = \frac{\exp(z(w, x)/\tau)}{\sum_d \exp(z(w, x)/\tau)}. \quad (3)$$

We formulate the local objective of FEDGKD for the k -th client at $t + 1$ round as

$$\min_w F_k(w) + \underbrace{\frac{\gamma}{2n_k}}_{\text{CE loss}} \sum_{i=1}^{n_k} \underbrace{\text{KL}(h_k(w_t, x_{ki}) || h_k(w, x_{ki}))}_{\text{KD loss}}, \quad (4)$$

Algorithm 1 The framework of FEDGKD

```

1: Notations. total number of clients  $K$ , total communication
   rounds  $T$ , local epochs  $E$ , fraction of participating clients
    $C$ , learning rate  $\eta$ , buffer size  $M$ ,  $\mathcal{B}$  is a set holding client's
   data sliced into batches of size  $B$ .

2: SERVEREXECUTION()  $\triangleright$ Server Model Aggregation
3: Initial global model  $w_0$ 
4: for each communication round  $t = 1, \dots, T$  do
5:    $K^t \leftarrow$  random sample a set of  $C \cdot K$  clients
6:   if FEDGKD-VOTE then
7:     Send the global models  $w_t, \dots, w_{t-M+1}$  to the
8:     selected clients
9:   else
10:    Send the global model  $w_t$  and ensemble global
11:    model  $\bar{w}_t$  to the selected clients
12:   end if
13:   Save global model  $w_t$  and ensemble the models
14:    $w_t, \dots, w_{t-M+1}$  in the buffer.
15: end for
16: for each client  $k \in K_t$  in parallel do
17:    $w_{t+1}^k \leftarrow$  CLIENTUPDATE( $k, w_t$ )
18:    $w_{t+1} \leftarrow \sum_{k=1}^K \frac{n_k}{n} w_{t+1}^k$ 
19: end for

20: CLIENTUPDATE( $k, w_t$ )  $\triangleright$ Local Model Training
21:  $w \leftarrow w_t$ 
22: for each local epoch  $i = 1, \dots, E$  do
23:   for each batch  $b \in \mathcal{B}$  do
24:      $w \leftarrow w - \eta \cdot \nabla \mathcal{L}(w, b)$   $\triangleright$ Update Local Weights via
       Eq. 5 or Eq. 6
25:   end for
26: end for
27: Return  $w$  back to server

```

where γ denotes the hyper-parameter to control the KD loss, n_k denotes the number of training samples for client k , $\mathbf{KL}$ denotes the Kullback-Leibler divergence. Optimizing the additional KL divergence loss between the outputs of global model w_t and local model w helps the global model to transfer its knowledge to the local model.

B. Historical Global Knowledge Ensemble

To preserve the information of historical active clients, we further ensemble the historical global models by averaging the models' parameters. The overall framework of FEDGKD is shown in Algorithm 1. To better utilize the recent models, we set a buffer on the server side. The global model w_t is collected from K participants at round t and stored in a buffer of size M on the server. We define $\bar{w}_t = \frac{1}{M} \sum_{m=1}^M w_{t-m+1}$ as the averaged weight of the global. In round $t+1$, M latest global models are averaged to a fused model $\bar{w}_t$ on the server, and the ensemble model is sent to sampled clients. Then the local model w_{t+1}^k is trained under the guidance of the latest M round global models to learn from the global knowledge. We define

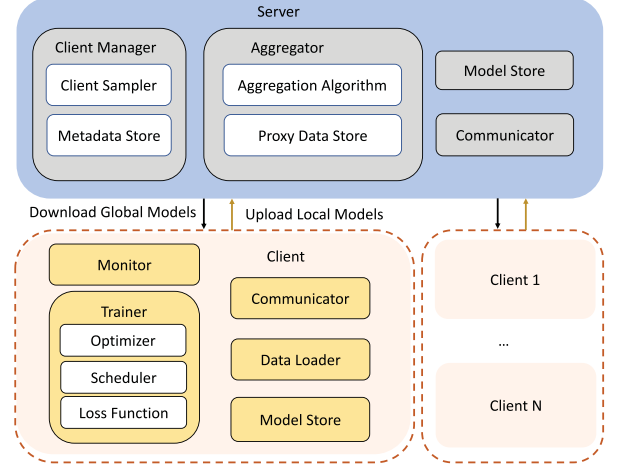


Fig. 5. System overview of FEDGKD. The system mainly consists of a server (coordinator) and multiple edge devices as clients.

the local objective function for the k -th client as

$$\min_w F_k(w) + \frac{\gamma}{2n_k} \sum_{i=1}^{n_k} \mathbf{KL}(h_k(\bar{w}_t, x_{ki}) || h_k(w, x_{ki})). \quad (5)$$

Note that there are many approaches to ensemble the models [43], [44], while most of them cannot be applied on the federated learning server due to the data requirement. We propose to ensemble the historical models simply by parameter averaging in the paper. If proxy data exists on the server, we can further improve the ensemble by knowledge distillation, and the improvement is shown in our experimental results when compared with FedDF [13].

We also develop FEDGKD-VOTE for ensembling past M models output on the client side, with the local objective defined as follows:

$$\min_w F_k(w) + \sum_{m=1}^M \frac{\gamma_m}{2n_k} \sum_{i=1}^{n_k} \mathbf{KL}(h_k(w_{t-m+1}, x_{ki}) || h_k(w, x_{ki})), \quad (6)$$

where $\gamma_1, \dots, \gamma_M$ are the distillation coefficients for historical global models.

For FEDGKD-VOTE, we set parameters $\gamma_1, \dots, \gamma_M$ by validation performance, following the calculation:

$$\frac{\gamma_i}{2} = \lambda \frac{e^{-\frac{L_i}{\beta}}}{\sum_i^M e^{-\frac{L_i}{\beta}}}, \quad (7)$$

where L_i is the validation loss of the i -th global model in the buffer, and β denotes the temperature and λ is the coefficient to control the scale. We set β as $\frac{1}{M}$ for simplicity.

C. System Design

We provide the system overview of FEDGKD in Fig. 5, which consists of a server and multiple edge device clients. The server mainly serves as a coordinator and does not require the client training data. The clients, mainly edge devices, are responsible for the main training tasks.

The server is composed of the following components.

- *Client Manager*: The client manager stores the metadata of each client, such as memory and network bandwidth. The server selects a subset of clients to participate in the FL training and distributes the model based on client metadata in each round. Furthermore, since clients have different computational capabilities, the server can configure models of different sizes for each client. In this paper, we mainly perform client selection through uniform distribution sampling.
- *Aggregator*: The aggregator is the core component of the server. In each communication round, the aggregator waits for all participants to upload their models and aggregates them based on different aggregation algorithms, the output of the aggregator is the global model to be sent to the clients. We implement different aggregation algorithms, with FedAvg being the primary aggregation algorithm used in FEDGKD. FEDGKD also supports FedADAM, which further improves upon FedAvg. If there is proxy data available, FEDGKD supports the heterogeneous models' aggregation algorithms like FedDF, and FEDGKD can further leverage proxy data for the historical model ensemble.
- *Model Store*: The server stores the best-performing historical global model based on its performance on the validation set in each round. For FEDGKD, it also stores the current round and the previous few rounds of global models.
- *Communicator*: The communicator is responsible for facilitating communication between the server and the clients. This includes broadcasting the global model to the clients and receiving their model updates. Additionally, it facilitates the transmission of additional information required by certain algorithms, such as generator and client label information for FedGen and teacher models for FEDGKD ($M > 1$). In our system, to ensure compatibility with CPU devices, we use MPI and Gloo as communication backends.

The client is composed of the following components:

- *Monitor*: It is responsible for monitoring device availability metrics, such as battery life and network bandwidth, to determine whether to terminate the current client training process.
- *Data Loader*: It is responsible for loading data onto the client, using multi-process parallelism to load data in batches from disk into memory. Prior to training, it also completes preprocessing tasks on the data, such as image cropping.
- *Trainer*: It is the core component of the client, adjusting the learning rate through the scheduler and executing gradient descent algorithms on the model using an optimizer. The loss function adjusts the model's training objective on the device. All regularization-based algorithms in the system are implemented through modifications to the loss function or optimizer.

In FEDGKD, the clients are stateless. FEDGKD does not store the states of the optimizer and scheduler, nor does FEDGKD need to store the historical local models and gradients, as

required by algorithms like FedDyn and MOON. Moreover, our system creates a model when the global model is distributed, and the clients can train different models in each round. The stateless design facilitates dynamic scaling of the system in the event of device failures. Besides, FEDGKD does not add any privacy risks. Unlike FedDistill and FedGen, we do not require the transmission of sensitive information from the client (label or logits information), and communication between the client and server only involves the models without any other sensitive information. Our system is compatible with many encryption algorithms.

D. FEDGKD Analysis

1) *Computation and Communication Overhead*: **Communication cost.** In Algorithm 1, we give a detailed description for FEDGKD and FEDGKD-VOTE. Both FEDGKD and FEDGKD-VOTE need to buffer the global models on the server, and FEDGKD ensembles the global models so that only $\bar{w}_t$ and w_t should be sent to the clients. FEDGKD increases the download communication cost compared to FedAvg if $M > 1$ and the same as FedAvg if $M = 1$. However, it substantially reduces the communication cost of FedAvg to reach target accuracy.

Computation overhead. For FEDGKD, the additional computation overhead comes from the inference time of the teacher model, so the computation overhead is close to FedProx. For MOON, the additional computation overhead comes from the global model's inference time and the previous local model's inference time, which is more than FEDGKD. For FedGen, the additional computation overhead comes from the random labels sampling and model's inference time, which is close to FEDGKD. For FEDGKD-VOTE, the computation overhead increases due to the number of teacher models growing. Besides, we can store the output logits of the teacher model in the first epoch of iterations on the disk, and load the logits for the rest of iterations. In this way, an epoch of the forward process is all the extra calculation overhead, and the computation overhead is close to FedDistill.

2) *Effectiveness Analysis*: As shown in Fig. 6, we compare the local model and the global model of FedAvg, FedProx, and FEDGKD at the last round trained on CIFAR-10 by visualizing the penultimate layer representations of the test dataset. As the features are clustered by classes in T-SNE, a good model (classifier) can give more clear separation in the feature points. We notice that in Fig. 6(a), the local model can barely classify the entire classes due to the non-IID data distribution. A clearer boundary can be observed in FEDGKD as shown in Fig. 6(d), where the improvement is brought by the knowledge distillation from the global ensemble model.

3) *Convergence Analysis*: Before formally stating the convergence result, we introduce the following notations. For any positive integer $a > 0$, denote $[a] = \{1, 2, \dots, a\}$. For any vector v , $[v]_j$ represents its j -th coordinate. To perform convergence analysis, assumptions made on the properties of the client functions and the Algorithm are summarized in Assumptions 1 and 2, respectively.

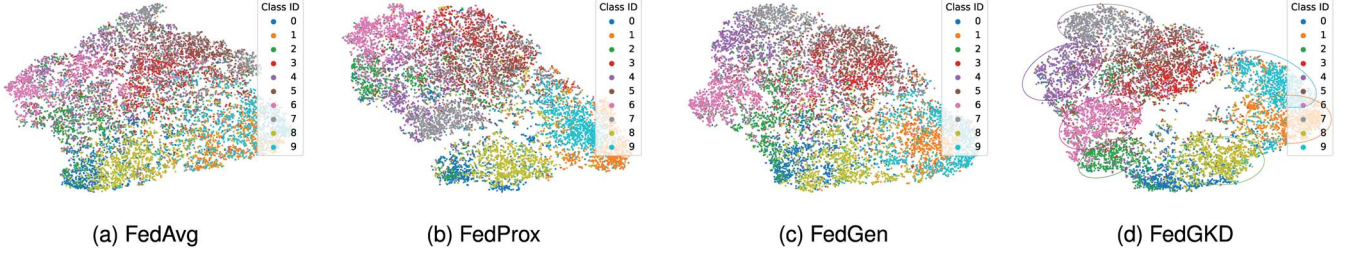


Fig. 6. t-SNE visualizations of feature of local models on CIFAR-10 test dataset (Model of 100 communication rounds under $\alpha = 0.1$ setting).

Assumption 1:

- 1) (Lower-bounded eigenvalue) For all $k \in [K]$, $F_k(\mathbf{w})$ is L-smooth, i.e., $\|\nabla F_k(\mathbf{w}) - \nabla F_k(\mathbf{w}')\| \leq L \|\mathbf{w} - \mathbf{w}'\|$ for some $L > 0$ and all $(\mathbf{w}, \mathbf{w}') \in \mathbb{R}^d \times \mathbb{R}^d$. Furthermore, we assume the minimal eigenvalue of the Hessian of the client loss function $\nabla^2 f_k(\mathbf{w})$ is uniformly bounded below by a constant $\lambda_{\min} \in \mathbb{R}$.
- 2) (Bounded dissimilarity) For all $k \in [K]$ and any $\mathbf{w} \in \mathbb{R}^d$, $\mathbb{E}_k[\|\nabla F_k(\mathbf{w})\|^2] \leq B^2 \|\nabla f(\mathbf{w})\|^2$, where the expectation is taken with respect to the client index k .
- 3) For all $k \in [K]$, h_k outputs a probabilistic vector, i.e., $[h_k(\cdot; \cdot)]_j \geq 0$ and $\sum_{j=1}^C [h_k(\cdot; \cdot)]_j = 1$ for all $j \in [C]$. Furthermore, we assume for any data point $x \in \mathbb{R}^d$, $\mathbf{w} \in \mathbb{R}^d$, and $k \in [K]$, there exists a constant $\delta > 0$ such that $\min_{j \in \{1, \dots, C\}} [h_k(\mathbf{w}, x)]_j \geq \delta > 0$. And there exists a constant $L_h > 0$, $|h_k(\mathbf{w}, x) - h_k(\mathbf{w}', x)| \leq L_h \|\mathbf{w} - \mathbf{w}'\|$ for any $(\mathbf{w}, \mathbf{w}') \in \mathbb{R}^d \times \mathbb{R}^d$.

We remark that Assumption 1.1 and Assumption 1.2 are also made in [15]. Assumption 1.3 is not stringent. For example, consider h_k being a neural network for a classification task. By the fact that the finite composition of Lipschitz continuous function gives the Lipschitz function, the last layer is 1-Lipschitz as long as other layers (see [45] for examples) are Lipschitz. Indeed F_k is already assumed to be L-smooth, which implies h_k is locally Lipschitz and we only require h_k to be Lipschitz along the iterate sequence generated by the algorithm.

Assumption 2: At the t -th round, the k -th client (if selected) solves the optimization problem $\mathbf{w}_{t+1}^k \approx \arg \min_{\mathbf{w} \in \mathbb{R}^d} m(\mathbf{w}; \mathbf{w}_t)$ approximately with

$$m_k(\mathbf{w}; \mathbf{w}_t) := F_k(\mathbf{w}) + \frac{\gamma}{2n_k} \sum_{i=1}^{n_k} \text{KL}(h_k(\mathbf{w}_t, x_{ki}) \| h_k(\mathbf{w}, x_{ki})),$$

such that

$$\|\nabla F_k(\mathbf{w}_{t+1}^k) + \frac{\gamma L_h}{\delta} (\mathbf{w}_{t+1}^k - \mathbf{w}_t)\| \leq \eta \|\nabla F_k(\mathbf{w}_t)\|, \quad (8)$$

where $\eta \in [0, 1)$ and $\gamma > 0$.

Lemma 3: The set $\{\mathbf{w} \in \mathbb{R}^d | \mathbf{w} \text{ satisfies Eq. (8)}\}$ is non-empty.

Proof: Note that

$$m_k(\mathbf{w}; \mathbf{w}_t) = F_k(\mathbf{w}) + \frac{\gamma}{2n_k} \sum_{i=1}^{n_k} \text{KL}(h_k(\mathbf{w}_t, x_{ki}) \| h_k(\mathbf{w}, x_{ki}))$$

$$\begin{aligned} &\leq F_k(\mathbf{w}) + \frac{\gamma}{2n_k} \sum_{i=1}^{n_k} \frac{\|h_k(\mathbf{w}_t, x_{ki}) - h_k(\mathbf{w}, x_{ki})\|^2}{\min_{j \in \{1, \dots, C\}} h_k(\mathbf{w}, x_{ki})} \\ &\leq F_k(\mathbf{w}) + \frac{\gamma L_h}{2\delta} \|\mathbf{w}_t - \mathbf{w}\|^2 \triangleq \tilde{m}_k(\mathbf{w}; \mathbf{w}_t), \end{aligned}$$

where the first inequality follows from [46, Lemma 11.6] and the second inequality follows from the Assumption 1.3. The Lemma 3 implies $\tilde{m}_k(\mathbf{w}; \mathbf{w}_t)$ uniformly upper bounds $m_k(\mathbf{w}; \mathbf{w}_t)$. Notice that for any approximate solution $\mathbf{w}_{t+1}^k$ satisfies $\tilde{m}(\mathbf{w}_{t+1}^k; \mathbf{w}_t) \leq \tilde{m}(\mathbf{w}^t; \mathbf{w}_t)$, then

$$m(\mathbf{w}_{t+1}^k; \mathbf{w}_t) \leq \tilde{m}(\mathbf{w}_{t+1}^k; \mathbf{w}_t) \leq \tilde{m}(\mathbf{w}^t; \mathbf{w}_t) = m(\mathbf{w}^t; \mathbf{w}_t),$$

which implies (8) will be satisfied at least by the minimizer of $\tilde{m}_k(\mathbf{w}, \mathbf{w}^t)$. $\square$

The Lemma 3 justifies the well-posedness of the Assumption 2. The following theorem gives the rate of convergence to a stationary point.

Theorem 4: Let Assumption 1 and Assumption 2 hold. Assume for each round, a subset of S_t clients are selected, with $|S_t| = S$, and the k -th client is selected with the probability p_k . If γ, η , and S are chosen to satisfy: 1) $\kappa := \frac{\gamma L_h}{\delta} + \lambda_{\min} > 0$; 2) $\rho > 0$, where

$$\rho = \frac{\delta}{\gamma L_h} \left(1 - \eta B - \frac{LB(1+\eta)}{\kappa} \right) - \frac{1}{\kappa} \left(\frac{\sqrt{2}B(1+\eta)}{\sqrt{S}} + \frac{L(1+\eta)^2 B^2}{2\kappa} + \frac{(2\sqrt{2}S+2)LB^2(1+\eta)^2}{\kappa S} \right),$$

then after T rounds,

$$\min_{t \in [T]} \mathbb{E}[\|\nabla f(\mathbf{w}^t)\|] \leq \frac{f(\mathbf{w}_0) - f(\mathbf{w}^*)}{\rho T}.$$

Proof: By Assumption 2, for the k -th client,

$$\begin{aligned} \nabla F_k(\mathbf{w}_{t+1}^k) + \frac{\gamma L_h}{\delta} (\mathbf{w}_{t+1}^k - \mathbf{w}_t) + e_{t+1}^k &= 0, \\ \|e_{t+1}^k\| &\leq \eta \|\mathbf{w}_t\|. \end{aligned}$$

Provided γ, η and S are chosen as described, then one can use the same proof in [15, Theorem 4] to show that

$$\mathbb{E}_{S_t}[f(\mathbf{w}_{t+1})] \leq f(\mathbf{w}_t) - \rho \|\nabla f(\mathbf{w}_t)\|^2.$$

Then take the total expectation with respect to all randomness and by telescoping, one reaches

$$\rho \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla f(\mathbf{w}_t)\|^2] \leq f(\mathbf{w}_0) - f(\mathbf{w}^*).$$

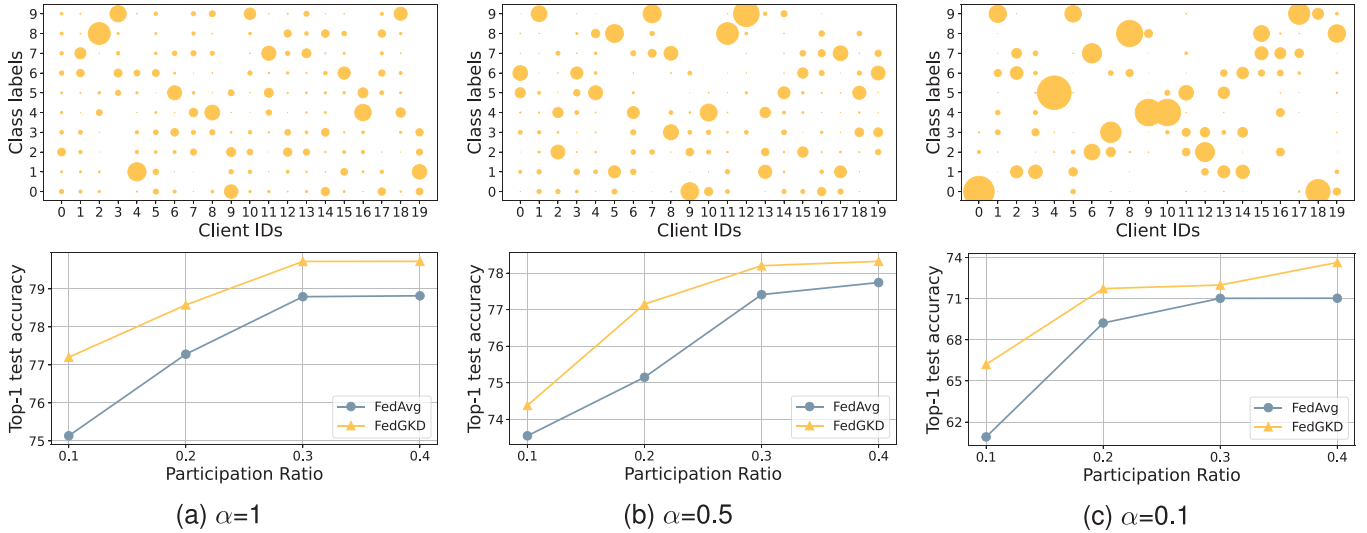


Fig. 7. Top: Visualization of the non-IID ness on CIFAR-10 dataset. Bottom: Performance of FedAvg and FedGKD ($M = 1$) with ResNet-8 on CIFAR-10 under different α and participation ratio setting.

Divide T on both sides, then

$$\begin{aligned} \min_{t \in [T]} \mathbb{E}[\|\nabla f(\mathbf{w}_t)\|] &\leq \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla f(\mathbf{w}_t)\|] \\ &\leq \frac{f(\mathbf{w}_0) - f(\mathbf{w}^*)}{\rho T}. \end{aligned}$$

Proof of Theorem 4.

Proof: By Assumption 2, for the k -th client,

$$\begin{aligned} \nabla F_k(\mathbf{w}_{t+1}^k) + \frac{\gamma L_h}{\delta} (\mathbf{w}_{t+1}^k - \mathbf{w}_t) + e_{t+1}^k &= 0 \\ \|e_{t+1}^k\| &\leq \eta \|\mathbf{w}_t\|. \end{aligned}$$

Provided γ, η and S are chosen as described, then one can use the same proof in [15, Theorem 4] to show that

$$\mathbb{E}_{S_t}[f(\mathbf{w}_{t+1})] \leq f(\mathbf{w}_t) - \rho \|\nabla f(\mathbf{w}_t)\|^2.$$

Then take the total expectation with respect to all randomness and by telescoping, one reaches

$$\rho \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla f(\mathbf{w}_t)\|]^2 \leq f(\mathbf{w}_0) - f(\mathbf{w}^*).$$

Divide T on both sides, then

$$\min_{t \in [T]} \mathbb{E}[\|\nabla f(\mathbf{w}_t)\|] \leq \frac{1}{T} \sum_{t=0}^{T-1} \mathbb{E}[\|\nabla f(\mathbf{w}_t)\|] \leq \frac{f(\mathbf{w}_0) - f(\mathbf{w}^*)}{\rho T}.$$

IV. EXPERIMENTS AND ANALYSIS

A. Experimental Setup

Datasets and models. We compare our approach with previous FL methods across both CV and NLP tasks. Specifically, we employ the ResNet architecture [47] for CV tasks and the

DistillBERT architecture [48] for NLP tasks. In the case of image classification tasks such as CIFAR-10 and CIFAR-100 [49], we utilize the ResNet-8 architecture. For the Tiny-ImageNet dataset, we opt for training the ResNet-50 architecture. It's worth noting that for all CV tasks, we use 90% of the data as the training dataset. Shifting to NLP tasks, we engage in text classification exercises involving two datasets: the SST-5 dataset for 5-class sentiment classification and the AG News dataset for 4-class news classification. Our training datasets consist of 50% and 100% of the available data for the respective tasks. The datasets corresponding to each task are provided in Table IV.

Data heterogeneity. Following prior work [13], [28], we use the Dirichlet distribution $\text{Dir}(\alpha)$ to sample disjoint non-IID client training data, where α denotes the concentration parameter, and the smaller α indicates higher data heterogeneity. Fig. 7 visualizes the sampling results on CIFAR-10 in our experiments.

Baselines. We compare our proposed methods with the following state-of-the-art approaches.

- **FedAvg:** FedAvg [5] is a classic federated learning algorithm, which directly uses averaging as an aggregation method.
- **FedProx:** FedProx [15] improves the local objective based on FedAvg. It adds the regularization of a proximal term for local training.
- **MOON:** MOON [19] employs contrastive learning by incorporating a projection layer into the original model architecture. In this approach, the local features of the previous model are regarded as negative samples, while the global model features serve as positive samples.
- **FedDistill⁺:** FedDistill [17] shares the logits of user data and adds the distillation regularization term for clients. We further develop it with model parameters sharing and name it FedDistill⁺.

TABLE II

TOP-1 TEST ACCURACY OVERVIEW GIVEN DIFFERENT DATA SETTINGS FOR RESNET ON CV DATASETS (20 CLIENTS WITH $C = 0.2$, 100 COMMUNICATION ROUNDS). **BOLD** NUMBER DENOTES THE BEST RESULT AND UNDERLINE DENOTES THE SECOND BEST RESULT

Dataset	Setting	FedAvg	FedProx	MOON	FedDistill ⁺	FedGen	FedDyn	FedADAM	FEDGKD	FEDGKD-VOTE	FEDGKD-PRJ
CIFAR-10	$\alpha=1$	77.30 $\pm$ 0.72	76.39 $\pm$ 0.59	78.17 $\pm$ 0.81	77.79 $\pm$ 0.51	76.44 $\pm$ 0.94	76.48 $\pm$ 0.51	77.60 $\pm$ 0.43	<u>79.14$\pm$0.72</u>	78.88 $\pm$ 0.39	79.45$\pm$0.37
	$\alpha=0.5$	75.15 $\pm$ 0.44	75.02 $\pm$ 0.74	77.24 $\pm$ 0.90	75.85 $\pm$ 0.63	73.62 $\pm$ 0.20	71.70 $\pm$ 0.96	71.59 $\pm$ 0.53	77.15 $\pm$ 0.28	77.44$\pm$0.60	77.38 $\pm$ 0.60
	$\alpha=0.1$	69.22 $\pm$ 0.91	68.79 $\pm$ 1.10	71.12 $\pm$ 0.69	67.23 $\pm$ 0.33	69.24 $\pm$ 0.32	66.81 $\pm$ 0.84	69.81 $\pm$ 0.37	72.27 $\pm$ 0.84	<u>72.99$\pm$0.49</u>	73.06$\pm$0.32
CIFAR-100	$\alpha=1$	42.22 $\pm$ 0.41	41.67 $\pm$ 0.56	40.76 $\pm$ 0.04	42.38 $\pm$ 0.24	39.83 $\pm$ 0.58	41.05 $\pm$ 0.32	42.28 $\pm$ 0.38	43.82$\pm$1.39	<u>42.91$\pm$0.23</u>	40.98 $\pm$ 0.41
	$\alpha=0.5$	40.42 $\pm$ 0.58	40.08 $\pm$ 0.24	35.89 $\pm$ 1.12	40.97 $\pm$ 0.12	39.30 $\pm$ 0.64	40.13 $\pm$ 0.35	39.27 $\pm$ 0.27	42.44$\pm$0.32	<u>42.34$\pm$0.31</u>	37.91 $\pm$ 0.63
	$\alpha=0.1$	34.91 $\pm$ 1.00	35.80 $\pm$ 0.15	31.90 $\pm$ 0.51	33.94 $\pm$ 0.45	33.13 $\pm$ 0.43	32.28 $\pm$ 0.32	34.48 $\pm$ 0.70	36.83 $\pm$ 0.20	37.29$\pm$0.07	33.42 $\pm$ 0.17
Tiny-ImageNet	$\alpha=1$	34.81 $\pm$ 1.57	36.54 $\pm$ 1.67	<u>43.20$\pm$0.66</u>	35.99 $\pm$ 1.18	34.79 $\pm$ 1.15	34.60 $\pm$ 0.73	31.45 $\pm$ 0.23	37.50 $\pm$ 0.43	35.79 $\pm$ 1.04	43.78$\pm$1.18
	$\alpha=0.5$	32.48 $\pm$ 1.59	35.41 $\pm$ 0.64	<u>40.16$\pm$1.96</u>	34.02 $\pm$ 0.73	34.47 $\pm$ 0.63	32.21 $\pm$ 0.45	30.29 $\pm$ 0.59	37.28 $\pm$ 0.86	37.35 $\pm$ 0.65	40.56$\pm$2.34
	$\alpha=0.1$	29.02 $\pm$ 0.65	31.93 $\pm$ 0.85	33.56 $\pm$ 0.16	28.95 $\pm$ 0.10	32.84 $\pm$ 0.99	29.72 $\pm$ 0.32	28.13 $\pm$ 0.46	<u>34.64$\pm$0.21</u>	33.71 $\pm$ 1.74	35.72$\pm$0.71

TABLE III

PERFORMANCE OF DISTILBERT GIVEN DIFFERENT DATA SETTINGS (4 PARTICIPANTS EACH ROUND, 10 COMMUNICATION ROUNDS)

Dataset	Setting	FedAvg	FedProx	MOON	FedDistill ⁺	FedGen	FEDGKD	FEDGKD-VOTE	FEDGKD-PRJ
AG News	$\alpha=0.1$	89.19 $\pm$ 0.87	89.19 $\pm$ 0.87	89.25 $\pm$ 0.48	89.35 $\pm$ 0.85	89.28 $\pm$ 0.77	89.60$\pm$0.44	89.04 $\pm$ 0.22	<u>89.48$\pm$0.18</u>
SST-5	$\alpha=0.1$	40.39 $\pm$ 0.50	40.42 $\pm$ 0.49	41.48 $\pm$ 2.24	41.25 $\pm$ 1.03	41.00 $\pm$ 3.23	43.24$\pm$1.80	<u>41.93$\pm$2.05</u>	41.21 $\pm$ 1.20

- **FedGen:** FedGen [18] shares the local label count information to train a global generator and regularizes the local training with this light-weighted generator.
- **FedDyn:** FedDyn [16] adds a dynamic regularization for local training.
- **FedADAM:** FedADAM [29] employs adaptive optimization for the server aggregation.

To compare with MOON, we add the projection layer for FEDGKD and develop it as FEDGKD-PRJ. Note that we find it fails to converge if Group Normalization (GN) is used in ResNet-50 and the projection is added, so we use Batch Normalization (BN) for ResNet-50 in MOON and FEDGKD-PRJ, to compare with MOON. Furthermore, we extend our analysis by comparing our approach with FedDF [13] across various CV tasks. The proxy data for CIFAR-10, CIFAR-100, and Tiny-ImageNet is set as CIFAR-100, ImageNet-32, CIFAR-100. Upon observing that FedDF does not exhibit competitiveness with the application of GN, we opt for BN as the normalization technique. To ensure a fair comparison with FedDF, we enhance FEDGKD into FEDGKD⁺ by incorporating proxy data and employing BN for normalization.

Implementation details. The proposed system and baselines are all implemented using PyTorch and evaluated on a Linux server equipped with four V100 GPUs. The Message Passing Interface (MPI) [50] serves as the communication backend, facilitating distributed computing. Due to the statistics differences in the mean and variance of each local data calculated by BN on heterogeneous datasets, which will lead to unstable training and decreased training accuracy [51], so we replace BN with GN. It is important to note that when GN is applied to ResNet-50 on Tiny-ImageNet, along with the addition of a 2-layer MLP, the model fails to converge. To compare with MOON [19], we use the BN on ResNet-50 in MOON and FEDGKD⁺.

For all the CV tasks, we set the channels of each group to 16. Client training utilizes the SGD optimizer, with learning rates fine-tuned from 0.1, 0.05, 0.01 for FedAvg, and a fixed learning rate of 0.05. Weight decay is maintained at 1×10^{-5} ,

coupled with a momentum value of 0.9. The CIFAR-10/100 datasets undergo 100 global communication rounds, while the Tiny-ImageNet dataset experiences 30 communication rounds. Local training spans 20 epochs, and the default participant ratio is established at $C = 0.2$. For all NLP fine-tuning tasks, Adam is used as the local optimizer, with a learning rate of 1×10^{-5} and zero weight decay. As fine-tuning typically requires a few epochs, communication rounds are set to 10. The local epoch is respectively set to 1 and 3 on AG News and SST-5. The clients who participated in each round are set to 4. Across all tasks, the training batch size remains consistent at $B = 64$. Our methods are subjected to three trials, and results are reported in terms of both mean and standard deviation.

B. Experimental Results

We compare the top-1 test accuracy of the different methods as reported in Tables II and III. Notably, our proposed method FEDGKD demonstrates superior performance over prior state-of-the-art models. When considering the $\alpha = 0.1$ setting for the Dirichlet distribution hyper-parameter, FEDGKD exhibits substantial improvements. Specifically, it outperforms FedAvg by 5.6% on Tiny-Imagenet, 1.9% on CIFAR-100, 3.0% on CIFAR-10, 0.4% on AG News, and 2.8% on SST-5. Furthermore, it is worth highlighting that FEDGKD introduces enhanced smoothness to the learning process, a quality evident when comparing it with FedAvg, as shown in Fig. 8. Consistently, FEDGKD maintains its competitive edge over FedAvg and FedProx.

Impact of data heterogeneity. We vary data heterogeneity across different levels, as evidenced by the results presented in Tables II and III. Among the compared approaches, namely FEDGKD, FEDGKD-VOTE, and FEDGKD-PRJ, it is clear that they outperform alternative methods, underscoring their robustness across varying α settings. Notably, FEDGKD demonstrates its superiority particularly as data heterogeneity escalates. In the case of FedProx, the performance closely approximates that of FedAvg when the coefficient μ is set to a small value; conversely, with larger values, convergence suffers, especially in

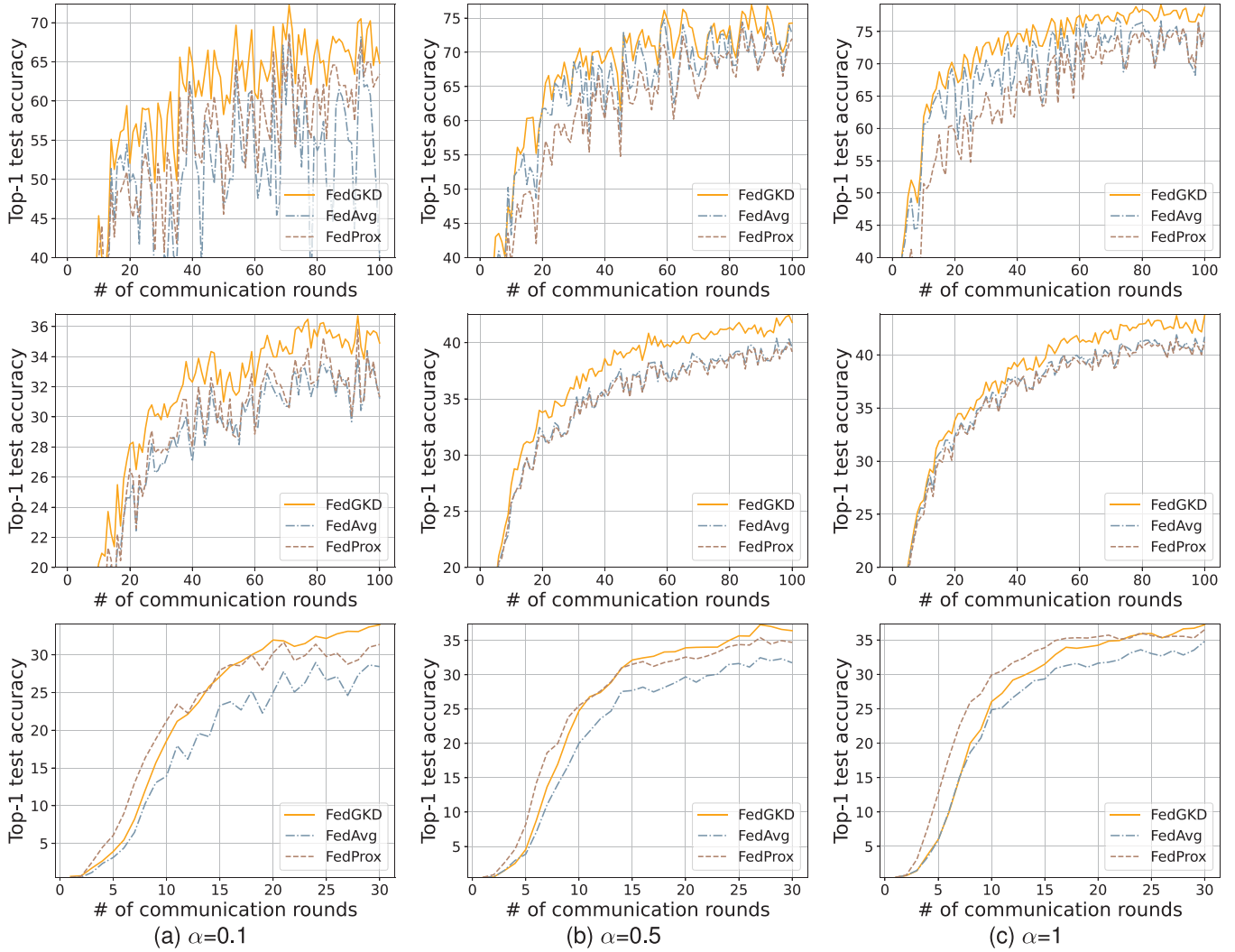


Fig. 8. Test accuracy of FedAvg, FedProx, and FEDGKD on CV datasets in the different number of communication rounds. From top to bottom, the dataset is CIFAR-10, CIFAR-100, Tiny-ImageNet. From left to right, the α ranges from 0.1 to 1.

scenarios of diminishing data heterogeneity. Conversely, in the context of FEDGKD, a regularization coefficient of 0.2 proves consistently beneficial in the majority of cases. FedGen, by leveraging a lightweight generator to primarily regularize the local model's classifier layer, inadvertently imposes limitations on its overall performance.

Impact of participation ratio. As shown in Table V, we present a more comprehensive exposition to showcase the advantages of our proposed method. As the number of participants grows, our proposed methods consistently outperform the alternative approaches. Notably, FEDGKD exhibits pronounced superiority over other methods, particularly when dealing with a smaller participant pool. In the case of FedGen, managing the generator training procedure becomes challenging under low participation ratios. For FedDistill⁺, the approach involves transmitting local logits from each client directly to the server, treating the averaged logits as global logits; however, the quality of these global logits is also compromised when participation ratios are low. Both FEDGKD and FEDGKD-VOTE leverage the

TABLE IV
STATISTICS OF TRAINING DATASETS

Dataset	Task	Train Data Size	Class	Clients
CIFAR-10	CV	45,000	10	20
CIFAR-100	CV	45,000	100	20
Tiny-Imagenet	CV	90,000	200	20
SST-5	NLP	8,544	5	10
AG News	NLP	60,000	4	20

ensemble and distillation of historical global models, resulting in enhanced precision of outcomes.

Robustness of the methods. As shown in Fig. 8, we can observe that the learning curve of FEDGKD consistently outperforms that of FedAvg and FedProx on CIFAR datasets. While FEDGKD initially exhibits poorer performance than FedProx on Tiny-Imagenet, it surpasses it as the communication rounds increase. Additionally, FEDGKD demonstrates a smoother learning curve in comparison to FedAvg. Our proposed method exhibits robustness across various communication rounds, as

TABLE V
TEST ACCURACY OF DIFFERENT FL METHODS IN DIFFERENT SAMPLING RATIO, WE EVALUATE ON CIFAR-10 WITH RESNET-8, $\alpha = 0.5$

Method	C=0.1		C=0.2		C=0.3		C=0.4	
	Best	Final	Best	Final	Best	Final	Best	Final
FedAvg	73.55±0.10	73.31±0.35	75.15±0.44	72.70±0.78	77.41±0.21	74.18±0.21	77.74±0.61	74.45±0.52
FedProx	72.59±0.66	72.59±0.66	75.02±0.74	72.08±0.92	76.36±0.64	74.55±0.48	76.65±0.76	74.03±0.61
MOON	73.47±0.47	72.57±0.31	77.24±0.90	74.48±1.01	77.88±0.26	75.00±0.33	78.20±0.57	75.05±0.44
FedDistill ⁺	73.75±0.35	73.51±0.35	75.85±0.63	73.63±0.26	77.31±0.65	74.38±0.19	78.22±0.45	74.92±0.47
FedGen	73.39±0.34	73.39±0.34	73.62±0.20	72.27±0.42	76.73±0.53	73.75±0.35	77.05±0.28	73.96±0.58
FEDGKD	74.38±1.11	74.10±0.86	77.15±0.28	74.27±0.33	78.20±0.77	75.45±0.34	78.32±0.40	75.39±0.67
FEDGKD-VOTE	75.41±0.56	74.25±0.58	77.44±0.60	73.96±0.86	77.72±0.51	75.74±0.31	78.08±0.58	75.75±0.75
FEDGKD-PRJ	75.16±0.67	74.46±0.54	77.38±0.60	74.55±0.78	78.02±0.45	74.54±0.42	78.28±0.16	75.24±0.33

TABLE VI
EVALUATION OF FL METHODS AT DIFFERENT COMMUNICATION ROUND ON CIFAR-10 WITH C = 0.2, $\alpha = 0.1$

Method	Test Accuracy at different communication round			
	10	20	50	100
FedAvg	36.30±1.24	49.54±0.99	46.47±1.67	40.09±2.29
FedProx	39.81±1.60	46.82±0.96	45.55±2.31	63.45±0.40
MOON	33.22±1.63	43.55±1.12	45.10±4.21	44.57±1.08
FedDistill ⁺	35.96±1.75	44.70±3.07	40.02±1.66	43.76±5.13
FedGen	37.87±0.40	51.58±1.65	47.66±0.97	45.25±3.05
FEDGKD	45.34±1.10	52.05±0.56	58.28±0.82	64.84±1.57
FEDGKD-VOTE	47.91±1.06	56.79±0.81	59.88±1.07	64.87±1.66
FEDGKD-PRJ	41.30±0.54	54.34±1.44	50.95±0.89	62.51±1.68

shown in Table VI. Many evaluation techniques experience a drop of over 20% in accuracy during the final communication round, as opposed to their peak accuracy (as shown in Table II), with the exception of FedProx and FEDGKD. Notably, both FEDGKD and FEDGKD-VOTE achieve higher accuracy while maintaining stability when compared to FedProx. This detailed comparison is provided in Fig. 6. Furthermore, in comparison to MOON which also learns from the global model, FEDGKD-PRJ demonstrates a more resilient training process.

Comments on projection layer. As illustrated in the paper of MOON [19], the projection head added before the classification layer is necessary for the method. We further conduct experiments involving the integration of the projection layer for both MOON and FEDGKD-PRJ. Surprisingly, our findings indicate that both models exhibit inferior performance compared to FedAvg on CIFAR-100, primarily due to the presence of the projection layer, as showcased in Table II. However, on CIFAR-10 and Tiny-Imagenet datasets, both MOON and FEDGKD-PRJ derive substantial benefits from the incorporation of the projection head, leading to a significant performance boost over alternative methods. Nonetheless, the FEDGKD-PRJ still outperforms MOON by 1.9%, 1.5%, and 2.1% on CIFAR-10, CIFAR-100, and Tiny-ImageNet, respectively, when $\alpha = 0.1$. In Table V, we note that as the participation ratio increases, the FEDGKD outperforms FEDGKD-PRJ, indicating that adding an MLP layer is not a general solution for all situations.

Effect of network architecture. We conduct experiments with different network architectures. The improvement for ResNet shows that our proposed method could work on modern networks and the superiority compared to FedAvg becomes more pronounced as the scale of the model grows, which is supported by results achieved on Tiny-Imagenet (utilizing

TABLE VII
COMPARISON WITH FEDDF BY USING ADDITIONAL PROXY SATA

Method	CIFAR-10	CIFAR-100	Tiny-ImageNet
FedDF	79.62±0.31	42.08±0.80	21.60±0.61
FEDGKD	77.15±0.28	42.44±0.32	37.28±0.86
FEDGKD ⁺	80.35±0.66	44.42±0.83	45.07±0.36

TABLE VIII
EFFECT OF DIFFERENT BUFFER LENGTH FOR FEDGKD, $C = 0.2$

Dataset	Buf. Len.	$\alpha = 1$	$\alpha = 0.5$	$\alpha = 0.1$
CIFAR-10	1	78.58±0.61	77.15±0.28	71.73±0.79
	3	78.63±0.19	76.91±0.05	72.12±0.41
	5	79.14±0.72	76.54±0.17	72.27±0.84
CIFAR-100	1	42.77±0.37	41.76±0.06	36.50±0.12
	3	42.92±0.19	42.33±0.21	36.82±0.24
	5	43.82±1.39	42.29±0.30	36.83±0.20
	7	43.15±1.37	42.44±0.32	36.65±0.64

ResNet-50). The efficacy of FedProx becomes evident in larger models due to its parameter regularization. In comparison to FedAvg, FedProx demonstrates a performance increase of 2.9%, while FEDGKD showcases improvements exceeding 2.7% over FedProx when $\alpha = 0.1$. When it comes to NLP tasks, even though FedAvg can readily achieve high performance based on pre-trained models, our proposed method consistently demonstrates its superiority in SST-5 results.

Comparison with FedDF. Table VII presents the comparison between FedDF and FEDGKD⁺. When analyzing the results of CIFAR-10 and CIFAR-100, we find that FEDGKD⁺ benefits from using proxy data and improves the performance of FedDF. Experimental results show that FEDGKD is orthogonal to different aggregation schemes. Specifically, on Tiny-ImageNet, FEDGKD⁺ modifies solely the normalization layer to maintain architectural consistency with FedDF, while adopting FedAvg for local model aggregation. Compared to FEDGKD⁺, FedDF experiences more than 23.4% performance drop, which is brought by the resolution difference of the proxy dataset (32×32) and training dataset (64×64). The significant performance drop reveals the limitation of FedDF.

C. Ablation Studies

Effect of buffer length setting. To investigate the best buffer length setting for FEDGKD, we conduct experiments on different buffer lengths, as shown in Tables VIII and IX. In

TABLE IX
PERFORMANCE OF DIFFERENT BUFFER LENGTH FOR
FEDGKD-VOTE ON CIFAR-10, $C = 0.2$

Buffer Length	$\alpha=1$	$\alpha=0.5$	$\alpha=0.1$
1	78.58 $\pm$ 0.61	77.15 $\pm$ 0.28	71.73 $\pm$ 0.79
3	78.48 $\pm$ 0.39	76.63 $\pm$ 0.40	72.27 $\pm$ 0.05
5	78.88$\pm$0.39	77.44$\pm$0.66	72.41$\pm$1.70
7	78.56 $\pm$ 0.63	76.78 $\pm$ 0.20	71.57 $\pm$ 1.43

TABLE X
PERFORMANCE OF DIFFERENT LOSS TYPES, UNDER
 $\alpha = 0.1$, $C = 0.2$ SETTING, BUFFER SIZE IS SET
TO $M = 1$

Loss Type	CIFAR-10	CIFAR-100	SST-5
None	69.22 $\pm$ 0.91	34.91 $\pm$ 1.00	40.39 $\pm$ 0.50
MSE	70.12 $\pm$ 0.98	38.19$\pm$0.77	43.50$\pm$1.68
KL	71.73$\pm$0.79	36.50 $\pm$ 0.12	43.24 $\pm$ 1.80

the case of FEDGKD, the communication cost remains constant regardless of the buffer size (M) due to the ensemble processing taking place on the server. Notably, when $M = 5$, the best performance is achieved for both $\alpha = 1$ and $\alpha = 0.1$, surpassing the performance of $M = 1$ by more than 0.5% on the CIFAR-10 dataset. For FEDGKD-VOTE, we find there is no more benefit by increasing the buffer length to 7, and it does not help significantly when the non-IID-ness is not a major issue or communication rounds are few.

Choice of distillation loss. In this paper, the KL-divergence is used as the distillation loss to control the discrepancy between the local logits and global logits distribution. We also consider using means squared error (MSE) defined over the logits as an alternative distillation loss. We evaluate the efficacy of the MSE loss on diverse datasets, namely CIFAR-10, CIFAR-100, and SST-5, while setting α to 0.1 and the buffer size M to 1. As shown in Table X, both choices outperform the FedAvg. However, when using the MSE loss for distillation, the accuracy is lower by 1.6% on CIFAR-10, comparable to SST-5, and 1.6% higher on CIFAR-100 in comparison to the accuracy achieved using KL-divergence loss. The convergence of the proposed algorithm is still guaranteed by the Lemma 3.

D. Efficiency Analysis

We examine the system metrics of FEDGKD and other baselines by evaluating the communication cost and computation overhead. The results are shown in Table XI. We evaluate the computation cost by running time on the client, and compute the communication cost of most methods as $2 \times (\#parameters) \times (\#participants) \times (\#communicationroundstoreachtargetaccuracy)$, where 2 denotes the downloading and uploading process. For FedGen, we add the generator downloading cost, and for FEDGKD ($M = 5$), the extra communication cost comes from the teacher model downloading.

From Table XI, it is evident that FEDGKD achieves a 50% reduction in communication rounds compared to FedAvg. Although FEDGKD requires downloading the teacher model when

TABLE XI
COMMUNICATION COST TO REACH TARGET TEST ACCURACY AND
COMPUTATION OVERHEAD PER ROUND ON CIFAR-10, $\alpha = 0.1$,
TARGET TEST ACCURACY IS 70%. CR: COMMUNICATION ROUNDS,
RT: RUNNING TIME

Method	CR	Cost (MB)	Client RT
FedAvg	132	20.60	28.76
FedADAM	113(-14.39%)	17.64(-14.39%)	28.76(+0.00%)
FedDistill ⁺	132(-0.00%)	20.60(-0.00%)	30.23(+5.1%)
FedProx	130(-1.50%)	20.29(-1.50%)	32.73(+13.80%)
FedDyn	200(+51.50%)	31.21(+51.50%)	33.09(+15.10%)
FedGen	108(-18.18%)	24.63(+19.56%)	32.92(+14.46%)
MOON	98(-25.76%)	15.30(-25.73%)	37.51(+30.42%)
FedGKD($M=1$)	74(-43.93%)	11.55(-43.93%)	32.49(+12.97%)
FedGKD($M=5$)	66(-50.00%)	15.45(-25.00%)	32.49(+12.97%)

$M > 1$, it still reduces the communication cost by 25% compared to FedAvg. Notably, FEDGKD can reach a comparable performance with less communication cost when $M = 1$. For FedGen, though it decreases the communication rounds to reach target accuracy, the generator downloading introduces additional communication costs. We find that MOON reaches the target performance with less communication cost, while the increased running time of MOON is twice that of FEDGKD. Based on the above discussions, if the system requires to reach a higher accuracy, FEDGKD should set a larger M . On the other hand, if the network bandwidth of the client is very limited, directly setting M to 1 is recommended.

In terms of the running time on the client side, we observe that FedADAM does not incur any additional client running time as it is an improvement on FedAvg implemented on the server side, which can be used in conjunction with FEDGKD. The running time of FEDGKD and regularization methods such as FedProx and FedGen are similar. Although FEDGKD slightly increases the client running time, considering that communication is the main bottleneck in federated learning [52], FEDGKD can still make a contribution to the federated learning system. It is worth noting that in our implementation, the forward propagation of the teacher model is executed in each iteration, while in practice, the running time of FEDGKD would be similar to that of FedDistill⁺ as the output logits of the teacher model in the first epoch are stored. The latter implementation has the potential to decrease computation time, albeit at the cost of requiring a greater amount of client disk storage space (calculated as the product of client data size and the dimensions of logits). Consequently, we recommend exercising adaptable decision-making, guided by the computational and storage capabilities of the client.

V. RELATED WORK

Federated learning. Earlier federated learning (FL) efforts primarily concentrated on independent and identically distributed (IID) data, as noted in [15], [53]. More recently, an increasing number of studies have shifted their attention towards addressing challenges posed by unbalanced, non-independent, and non-identically distributed (non-IID) data, which are inherent in real-world scenarios [5], [27]. At the practical level, FL has found widespread application across

various domains, including healthcare [2], next-word prediction [5], [54], natural language processing [55], and more. Notably, recent research has placed a special emphasis on resource control in the context of federated learning [5], [9]. The other work [56] studies LDP on federated learning problem. However, as we discussed in the experiments part, both of them struggle to attain favorable performance within the confines of a restricted privacy budget.

Knowledge distillation. Knowledge distillation (KD) [37] is originally proposed for model compression by mimicking the predictions of the target large deep neural network. Numerous studies are focused on the task of imparting the collective knowledge of multiple teachers to a single student model [57], [58]. In addition to teacher-student learning, knowledge distillation is also extended to other learning and tasks [37], [59], such as mutual learning, assistant teaching, lifelong learning, and self-learning. Self-distillation is first proposed in [60] and has received significant interest recently. Noteworthy methodologies like Temporal Ensembling [20] and Mean Teacher [21] have emerged as semi-supervised learning techniques, designed to imbue the model with self-ensemble regularization.

Knowledge distillation in federated learning. Knowledge distillation has recently been demonstrated as an efficient method to address the challenges posed by FL [13], [17], [18]. To tackle the data heterogeneity, some approaches [13], [17] use KD to ensemble the local models. Particularly, FedDF [13] uses the averaged logits of the local model for aggregation. To combat the problem of local forgetting, FedNTD [61] introduces the concept of Not-True Distillation. FedDistill [17] performs KD to refine the global model by sharing local logits. FedAUX [62] expands the idea of federated distillation to conduct self-supervised pre-training. FedBE [14] introduces the Bayesian view on sampling global model for aggregation. FedGen [18] learns a global generator to aggregate the local information and distills global knowledge to users. In another part, KD can also release the communication limitation.

In general, previous works require either additional local information [17] or auxiliary data [13], [18]. However, as discussed in [15], [26], the partial global sharing of local information violates privacy, and using globally-shared auxiliary data should be cautiously generated or collected. Compared with previous methods, our approach does not require any local information or auxiliary data.

VI. CONCLUSION

In this paper, we introduce an innovative global knowledge distillation approach, named FEDGKD, which leverages insights gained from previous global models to effectively address the challenge of client drift. FEDGKD makes the local model's feature representation more comprehensive by utilizing knowledge distillation, and ensembles the global models to produce a more accurate model. We validate the efficacy of the proposed methods on CV and NLP datasets under different non-IID settings. The experimental results show that FEDGKD indeed improves the performance in the non-IID federated settings.

REFERENCES

- [1] M. S. Elbamby et al., "Wireless edge computing with latency and reliability guarantees," *Proc. IEEE*, vol. 107, no. 8, pp. 1717–1737, Aug. 2019.
- [2] W. Y. B. Lim et al., "Decentralized edge intelligence: A dynamic resource allocation framework for hierarchical federated learning," *IEEE Trans. Parallel Distrib. Syst.*, vol. 33, no. 3, pp. 536–550, Mar. 2022.
- [3] A. Menshchikov et al., "Real-time detection of hogweed: UAV platform empowered by deep learning," *IEEE Trans. Comput.*, vol. 70, no. 8, pp. 1175–1188, Aug. 2021.
- [4] M. Tsukada, M. Kondo, and H. Matsutani, "A neural network-based on-device learning anomaly detector for edge devices," *IEEE Trans. Comput.*, vol. 69, no. 7, pp. 1027–1044, Jul. 2020.
- [5] J. Zhang et al., "Adaptive federated learning on non-IID data with resource constraint," *IEEE Trans. Comput.*, vol. 71, no. 7, pp. 1655–1667, Jul. 2022.
- [6] D. Huba et al., "PAPAYA: Practical, private, and scalable federated learning," in *Proc. MLSys*, 2022, pp. 814–832.
- [7] A. Nedić, A. Olshevsky, and M. G. Rabbat, "Network topology and communication-computation tradeoffs in decentralized optimization," *Proc. IEEE*, vol. 106, no. 5, pp. 953–976, May 2018.
- [8] D. Li, Z. Hu, Z. Lai, Y. Zhang, and K. Lu, "Coordinative scheduling of computation and communication in data-parallel systems," *IEEE Trans. Comput.*, vol. 70, no. 12, pp. 2182–2197, Dec. 2021.
- [9] Y. Zhan, P. Li, L. Wu, and S. Guo, "L4L: Experience-driven computational resource control in federated learning," *IEEE Trans. Comput.*, vol. 71, no. 4, pp. 971–983, Apr. 2022.
- [10] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *Proc. ICLR*, 2018.
- [11] S. P. Karimireddy, S. Kale, M. Mohri, S. Reddi, S. Stich, and A. T. Suresh, "SCAFFOLD: Stochastic controlled averaging for on-device federated learning," in *Proc. ICML*, 2020, pp. 5132–5143.
- [12] W. Wu, L. He, W. Lin, R. Mao, C. Maple, and S. A. Jarvis, "SAFA: A semi-asynchronous protocol for fast federated learning with low overhead," *IEEE Trans. Comput.*, vol. 70, no. 5, pp. 655–668, May 2021.
- [13] T. Lin, L. Kong, S. U. Stich, and M. Jaggi, "Ensemble distillation for robust model fusion in federated learning," in *Proc. NeurIPS*, 2020, pp. 2351–2363.
- [14] H.-Y. Chen and W.-L. Chao, "FedBE: Making Bayesian model ensemble applicable to federated learning," in *Proc. ICLR*, 2021.
- [15] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated optimization in heterogeneous networks," in *Proc. MLSys*, 2020, pp. 429–450.
- [16] D. A. E. Acar, Y. Zhao, R. Matas, M. Mattina, P. Whatmough, and V. Saligrama, "Federated learning based on dynamic regularization," in *Proc. ICLR*, 2020.
- [17] H. Seo, J. Park, S. Oh, M. Bennis, and S.-L. Kim, "Federated knowledge distillation," 2020, *arXiv:2011.02367*.
- [18] Z. Zhu, J. Hong, and J. Zhou, "Data-free knowledge distillation for heterogeneous federated learning," in *Proc. ICML*, 2021, pp. 12878–12889.
- [19] Q. Li, B. He, and D. Song, "Model-contrastive federated learning," in *Proc. CVPR*, 2021, pp. 10713–10722.
- [20] L. Samuli and A. Timo, "Temporal ensembling for semi-supervised learning," in *Proc. ICLR*, 2017.
- [21] A. Tarvainen and H. Valpola, "Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results," in *Proc. NeurIPS*, 2017, pp. 1195–1204.
- [22] K. Kim, B. Ji, D. Yoon, and S. Hwang, "Self-knowledge distillation with progressive refinement of targets," in *Proc. CVPR*, 2021, pp. 6567–6576.
- [23] F. Lai, X. Zhu, H. V. Madhyastha, and M. Chowdhury, "Oort: Efficient federated learning via guided participant selection," in *Proc. OSDI*, 2021, pp. 19–35.
- [24] E. Diao, J. Ding, and V. Tarokh, "HeteroFL: Computation and communication efficient federated learning for heterogeneous clients," in *Proc. ICLR*, 2021.
- [25] R. Dai, L. Shen, F. He, X. Tian, and D. Tao, "DisPFL: Towards communication-efficient personalized federated learning via decentralized sparse training," in *Proc. ICML*, 2022, pp. 4587–4604.
- [26] K. Bonawitz et al., "Practical secure aggregation for privacy-preserving machine learning," in *Proc. CCS*, 2017, pp. 1175–1191.
- [27] F. Sattler, S. Wiedemann, K.-R. Müller, and W. Samek, "Robust and communication-efficient federated learning from non-i.i.d. data," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 31, no. 9, pp. 3400–3413, Sep. 2020.

- [28] T.-M. H. Hsu, H. Qi, and M. Brown, "Measuring the effects of non-identical data distribution for federated visual classification," 2019, *arXiv:1909.06335*.
- [29] S. J. Reddi et al., "Adaptive federated optimization," in *Proc. ICLR*, 2020.
- [30] H. Wang, M. Yurochkin, Y. Sun, D. Papailiopoulos, and Y. Khazaeni, "Federated learning with matched averaging," in *Proc. ICLR*, 2020.
- [31] T. Yoon, S. Shin, S. J. Hwang, and E. Yang, "FedMix: Approximation of mixup under mean augmented federated learning," in *Proc. ICLR*, 2020.
- [32] L. Zhang, L. Shen, L. Ding, D. Tao, and L.-Y. Duan, "Fine-tuning global model via data-free knowledge distillation for non-iid federated learning," in *Proc. CVPR*, 2022, pp. 10174–10183.
- [33] X. Li, M. Jiang, X. Zhang, M. Kamp, and Q. Dou, "FedBN: Federated learning on non-IID features via local batch normalization," in *Proc. ICLR*, 2021.
- [34] A. Vaswani et al., "Attention is all you need," in *Proc. NeurIPS*, 2017, pp. 5998–6008.
- [35] Y. Tan et al., "FedProto: Federated prototype learning across heterogeneous clients," in *Proc. AAAI*, 2022, pp. 8432–8440.
- [36] F. Chen, G. Long, Z. Wu, T. Zhou, and J. Jiang, "Personalized federated learning with a graph," in *Proc. IJCAI*, 2022, pp. 2575–2582.
- [37] S. I. Mirzadeh, M. Farajtabar, A. Li, N. Levine, A. Matsukawa, and H. Ghasemzadeh, "Improved knowledge distillation via teacher assistant," in *Proc. AAAI*, 2020, pp. 5191–5198.
- [38] D. Li and J. Wang, "FedMD: Heterogenous federated learning via model distillation," 2019, *arXiv:1910.03581*.
- [39] C. He, M. Annavaram, and S. Avestimehr, "Group knowledge transfer: Federated learning of large CNNs at the edge," in *Proc. NeurIPS*, 2020, pp. 14068–14080.
- [40] Q. Li, B. He, and D. Song, "Practical one-shot federated learning for cross-silo setting," in *Proc. IJCAI*, 2021, pp. 1484–1490.
- [41] Z. Allen-Zhu and Y. Li, "Towards understanding ensemble, knowledge distillation and self-distillation in deep learning," in *Proc. ICLR*, 2022.
- [42] G. Hinton, O. Vinyals, and J. Dean, "Distilling the knowledge in a neural network," 2015, *arXiv:1503.02531*.
- [43] X. Lan, X. Zhu, and S. Gong, "Knowledge distillation by on-the-fly native ensemble," in *Proc. NeurIPS*, 2018, pp. 7528–7538.
- [44] D. Walawalkar, Z. Shen, and M. Savvides, "Online ensemble model compression using knowledge distillation," in *Proc. ECCV*. Cham, Germany: Springer, 2020, pp. 18–35.
- [45] A. Virmaux and K. Scaman, "Lipschitz regularity of deep neural networks: Analysis and efficient estimation," in *Proc. NeurIPS*, 2018.
- [46] J. S. Klemelä, *Smoothing of Multivariate Data: Density Estimation and Visualization*. Hoboken, NJ, USA: Wiley, 2009.
- [47] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," in *Proc. CVPR*, 2016, pp. 770–778.
- [48] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: Smaller, faster, cheaper and lighter," 2019, *arXiv:1910.01108*.
- [49] A. Krizhevsky, "Learning multiple layers of features from tiny images," Master's thesis, Univ. Toronto, Canada, 2009.
- [50] H. Dai, X. Peng, X. Shi, L. He, Q. Xiong, and H. Jin, "Reveal training performance mystery between TensorFlow and PyTorch in the single GPU environment," *Sci. China Inf. Sci.*, vol. 65, no. 1, 2022.
- [51] K. Hsieh, A. Phanishayee, O. Mutlu, and P. Gibbons, "The non-IID data quagmire of decentralized machine learning," in *Proc. ICML*, 2020, pp. 4387–4398.
- [52] P. Kairouz et al., "Advances and open problems in federated learning," *Found. Trends® Mach. Learn.*, vol. 14, no. 1–2, pp. 1–210, 2021.
- [53] K. Bonawitz et al., "Towards federated learning at scale: System design," in *Proc. MLSys*, 2019.
- [54] H. B. McMahan, D. Ramage, K. Talwar, and L. Zhang, "Learning differentially private recurrent language models," in *Proc. ICLR*, 2018.
- [55] Y. Tian, Y. Wan, L. Lyu, D. Yao, H. Jin, and L. Sun, "FedBERT: When federated learning meets pre-training," *ACM Trans. Intell. Syst. Technol.*, vol. 13, no. 4, pp. 66:1–66:26, 2022.
- [56] S. Truex, L. Liu, K.-H. Chow, M. E. Gursoy, and W. Wei, "LDP-Fed: Federated learning with local differential privacy," in *Proc. 3rd ACM Int. Workshop Edge Syst., Anal. Netw.*, 2020.
- [57] J. Hamm, Y. Cao, and M. Belkin, "Learning privately from multiparty data," in *Proc. ICML*, 2016, pp. 555–563.
- [58] N. Papernot, M. Abadi, U. Erlingsson, I. Goodfellow, and K. Talwar, "Semi-supervised knowledge transfer for deep learning from private training data," in *Proc. ICLR*, 2017.
- [59] Y. Zhang, T. Xiang, T. M. Hospedales, and H. Lu, "Deep mutual learning," in *Proc. CVPR*, 2018, pp. 4320–4328.
- [60] T. Furlanello, Z. Lipton, M. Tschannen, L. Itti, and A. Anandkumar, "Born again neural networks," in *Proc. ICML*, 2018, pp. 1602–1611.
- [61] G. Lee, M. Jeong, Y. Shin, S. Bae, and S.-Y. Yun, "Preservation of the global knowledge by not-true self knowledge distillation in federated learning," in *Proc. NIPS*, 2022, pp. 38461–38474.
- [62] F. Sattler, T. Korjakow, R. Rischke, and W. Samek, "FedAUX: Leveraging unlabeled auxiliary data in federated learning," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 9, pp. 5531–5543, Sep. 2023.



learning, and distributed optimization.



Dezhong Yao (Member, IEEE) received the Ph.D. degree in computer science from the Huazhong University of Science and Technology (HUST), Wuhan, China, in 2016. He was a Research Fellow with Nanyang Technological University, Singapore, between 2016 and 2019, and was a Visiting Scholar with the Carnegie Mellon University, Pittsburgh, PA, USA, between 2010 and 2012. He is currently an Associate Professor with the School of Computer Science and Technology, HUST. His research interests include large-scale machine learning, federated learning, and distributed optimization.

Wanning Pan received the B.S. degree from the Huazhong University of Science and Technology, Wuhan, China, in 2020. She is working toward the master's degree with the Huazhong University of Science and Technology, Wuhan, China. Her main research interests include data heterogeneity and system heterogeneity issues in federated learning.

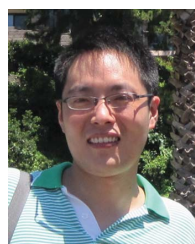


Yutong Dai is currently working toward the Ph.D. degree in industrial and systems engineering with Lehigh University, USA. His main research interests include designing, analyzing, and implementing algorithms for large-scale nonconvex nonsmooth optimization problems that arise in machine learning and federated learning.



programming languages, software engineering, and data mining.

Yao Wan received the Ph.D. degree from the College of Computer Science, Zhejiang University, Hangzhou, China, in 2019. He is currently a Lecturer with the College of Computer Science and Technology, Huazhong University of Science and Technology. He has been a Visiting Student with the University of Technology Sydney and the University of Illinois at Chicago, in 2016 and 2018, respectively. His research interests include the synergy between artificial intelligence and software engineering, especially natural language processing, programming languages, software engineering, and data mining.



Xiaofeng Ding received the Ph.D. degree in computer science from the Huazhong University of Science and Technology, Wuhan, China, in 2009. He is currently working as a Professor with the School of Computer Science and Technology, Huazhong University of Science and Technology, Wuhan, China. His research interests mainly include data privacy and query processing, data encryption, graph databases, and crowdsourcing.



Technology, Wuhan, where he is currently a full Professor and a Special Research Fellow, working in the areas of wireless sensor networks, ubiquitous computing, and green communications.



Excellent Youth Award from the National Science Foundation of China in 2001. He is a fellow of CCF and a life member of the ACM. He has co-authored more than 20 books and published over 900 research papers. His research interests include computer architecture, parallel and distributed computing, big data processing, data storage, and system security.

Chen Yu received the B.S. degree in mathematics and the M.S. degree in computer science from Wuhan University, Wuhan, China, in 1998 and 2002, respectively, and the Ph.D. degree in information science from Tohoku University, Sendai, Japan, in 2005. From 2005 to 2006, he was a Japan Science and Technology Agency Postdoctoral Researcher with the Japan Advanced Institute of Science and Technology. Since 2008, he has been with the School of Computer Science and Technology, Huazhong University of Science and



federated learning.

Zheng Xu received the B.Eng. and M.Eng. degrees from the Department of Electronic Engineering and Information Science, University of Science and Technology of China (USTC), and the Ph.D. degree from the Department of Computer Science, University of Maryland, College Park, under the supervision of Dr. Tom Goldstein, in 2019. Before that, he was a Project Officer with Visual Computing Research Group, School of Computer Engineering, Nanyang Technological University, Singapore. He is a Senior Research Scientist at Google working on



ON INDUSTRIAL INFORMATICS, IEEE TRANSACTIONS ON MOBILE COMPUTING, and IEEE TRANSACTIONS ON NEURAL NETWORKS AND LEARNING SYSTEMS.

Lichao Sun received the Ph.D. degree from the University of Illinois at Chicago, Chicago, IL, USA, in 2020. He is currently an Assistant Professor with Lehigh University, Bethlehem, PA, USA. His research interests include federated learning, reinforcement learning, and deep learning. He has studied various computer vision, natural language processing, and graph applications. He has published more than 40 papers in top-tier conferences and journals, such as NeurIPS, KDD, AAAI, IJCAI, ICLR, CCS, Usenix-Security, IEEE TRANSACTIONS